

Stroj s kroglicami

Madina je izumila stroj, ki zabava udeležence IOI. Notranja zgradba stroja je naslednja:

- Stroj sestavlja N **vozlišč**, oštevilčenih od 0 do $N - 1$. Vozlišče $N - 1$ imenujemo **koren**.
- Za vsako vozlišče u , kjer $0 \leq u < N - 1$, je **starš** vozlišča u neko vozlišče $P[u]$, kjer $P[u] > u$. Vozlišče u je **otrok** $P[u]$. Koren nima starša. Vozlišče brez otrok imenujemo **list**.

Ne poznate vrednosti N in **ne** poznate starša $P[u]$. Namesto tega vam Madina pove M , število listov, in da so indeksi listov $0, 1, \dots, M - 1$. Vaša naloga je ugotoviti notranjo zgradbo stroja, tako da ga upravljate.

Vsako vozlišče stroja lahko drži največ eno **kroglico**, kjer ima vsaka kroglica neko **vrednost**, ki je nenegativno celo število. Vozlišče ima vrednost x , če drži kroglico z vrednostjo x . Vozlišče je **zasedeno**, če vsebuje kroglico, sicer je **prosto**. Na začetku so vsa vozlišča prosta.

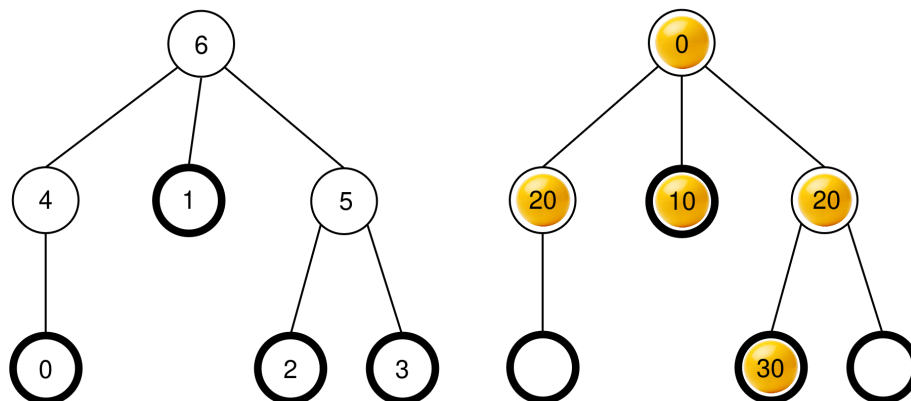
Izvajate lahko dve vrsti operacij:

- **insert**(U, X): poskusite vstaviti novo kroglico z vrednostjo X na list U .
 - Če je list U zaseden, operacija vrne `false` in ne vstavi kroglice.
 - Če je list U prost, se kroglica postavi na vozlišče U . Nato se kroglica večkrat premakne na starša trenutnega vozlišča, vse dokler je njegov starš prost. Premikanje se ustavi, ko kroglica doseže koren ali vozlišče, katerega starš je zaseden. Operacija vrne `true`.
- **collect**(): če je koren prost (kar pomeni, da je stroj prazen), `collect` vrne prazno polje. V nasprotnem primeru rekurzivna procedura `traverse`, ki je opisana spodaj, zbere vrednosti vseh kroglic, ki so trenutno v stroju, v polje S . Sprva je polje S prazno in se pokliče `traverse(N - 1)`.

```
traverse(u):  
    dodaj vrednost kroglice na vozlišču u na konec S  
    naj bo c[u] seznam zasedenih otrok u  
    uredi c[u] naraščajoče glede na vrednosti kroglic na teh vozliščih  
    (če imajo vozlišča enako vrednost, se lahko pojavijo v poljubnem vrstnem redu)  
    za vsak v v c[u]:  
        traverse(v)
```

Po končanju te metode, se **vse kroglice odstranijo** iz stroja in `collect` vrne S .

Na primer, vzemimo stroj, ki ga sestavlja $N = 7$ vozlišč in $P = [4, 6, 5, 5, 6, 6]$. Spodnja slika na levi strani prikazuje prazen stroj z indeksiranimi vozlišči. Na desni strani je stanje stroja po izvedbi določenega zaporedja operacij `insert` (to zaporedje je podano v razdelku Primer).



Ko pokličemo `collect()`, je koren (vozlišče 6) zaseden, zato se S inicializira kot $S = []$ in se pokliče `traverse(6)`.

`traverse(6)`: vozlišče 6 ima vrednost 0. Če to vrednost dodamo S , dobimo $S = [0]$. Otroci vozlišča 6 so vozlišča 4, 1 in 5, ki so vsi zasedeni. Njihove vrednosti so 20, 10 in 20. Če jih uredimo v naraščajočem vrstnem redu, dobimo $c[6] = [1, 5, 4]$ (opomba: $c[6] = [1, 4, 5]$ bi tudi bila veljavna rešitev, saj imata vozlišči 4 in 5 enaki vrednosti). Procedura nato pokliče `traverse` na vsakem vozlišču v $c[6]$ v tem vrstnem redu.

- **`traverse(1)`**: vozlišče 1 ima vrednost 10. Če to vrednost dodamo S , dobimo $S = [0, 10]$. Vozlišče 1 nima zasedenih otrok, zato se ta klic zaključí.
- **`traverse(5)`**: vozlišče 5 ima vrednost 20. Če to vrednost dodamo S , dobimo $S = [0, 10, 20]$. Vozlišče 5 ima enega zasedenega otroka, vozlišče 2, zato je $c[5] = [2]$ in procedura pokliče `traverse(2)`.
 - **`traverse(2)`**: vozlišče 2 ima vrednost 30. Če to vrednost dodamo S , dobimo $S = [0, 10, 20, 30]$. Vozlišče 2 nima zasedenih otrok, zato se ta klic zaključí.
 - Izvajanje se vrne k `traverse(5)`, kjer so pregledani vsi otroci $c[5]$, zato se klic zaključí.
- **`traverse(4)`**: vozlišče 4 ima vrednost 20. Če to vrednost dodamo S , dobimo $S = [0, 10, 20, 30, 20]$. Vozlišče 4 nima zasedenih otrok, zato se ta klic zaključí.

Vsi klici so končani in ni več vozlišč za pregled. Na koncu se odstranijo vse kroglice stroja in `collect` vrne $S = [0, 10, 20, 30, 20]$.

Vaša naloga je ugotoviti število vozlišč N in poročati o polju staršev $R = [R[0], R[1], \dots, R[N-2]]$, ki opisuje zgradbo stroja, vendar z morebitno drugačnim označevanjem vozlišč, ki niso listi in niso koren. Natančneje, poročano polje R velja za pravilno, če je mogoče vsakemu vozlišču u tega stroja dodeliti različno oznako $L[u]$, pri čemer $0 \leq L[u] < N$, tako da velja:

- $L[u] = u$ za vse $0 \leq u < M$ in $u = N - 1$ in
- $L[P[u]] = R[L[u]]$ za vse $0 \leq u < N - 1$.

Ni zahtevano, da $R[u] > u$. Stroj **mora biti prazen**, ko poročate vašo rešitev.

Naj bo K število opravljenih operacij `collect`, B pa največja vrednost katerekoli kroglice med vsemi klici `insert`. Naj bo $C = K + B$. Potem C **ne sme preseči** 1000. Vaša ocena pri nekaterih podnalogah je odvisna od vrednosti C .

Podrobnosti implementacije

Implementirajte naslednjo funkcijo:

```
std::vector<int> find_structure(int M)
```

- M : število listov v stroju.
- Funkcijo se kliče natanko enkrat za testni primer.
- Funkcija mora vrniti polje $R = [R[0], R[1], \dots, R[N-2]]$ dolžine $N-1$, kjer je R pravilno polje staršev.

Za upravljanje stroja lahko vaša funkcija kliče naslednji funkciji.

Prva funkcija je:

```
bool insert(int U, int X)
```

- U : indeks lista, kamor naj bo vstavljena kroglica. Pogoj $0 \leq U < M$ mora biti izpolnjen.
- X : celoštevilka vrednost kroglice. Pogoj $0 \leq X \leq 1000$ mora biti izpolnjen.
- Ta funkcija vrne `true`, če je bila kroglica uspešno vstavljena, oziroma `false`, če je bil list U že zaseden.
- To funkcijo lahko pokličete največ \$500\backslash.000\$ krat.

Druga funkcija je:

```
std::vector<int> collect()
```

- Pobere vrednosti vseh vstavljenih kroglic, izprazni stroj in vrne polje S .

Če kadarkoli med izvajanjem vaše rešitve vrednost C preseže 1000, bo vaša rešitev prejela razsodbo `Output isn't correct: Too many resources used` (Odgovor je napačen: preveč porabljenih virov).

Struktura stroja je določena pred klicem `find_structure`. Ocenjevalnik je **determinističen** v tem smislu, da, če ga poženete dvakrat in obe izvedbi izvedeta enako zaporedje operacij, bodo klici `collect` vedno vrnili enaka polja.

Omejitve

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Podnaloge

Podnaloga	Točke	Dodatne omejitve
1	5	Koren ima natanko M otrok.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Brez dodatnih omejitev.

V podnalogah 3 in 4 je vaša ocena odvisna od vrednosti C , kot sledi.

Podnaloga 3 (25 točk)

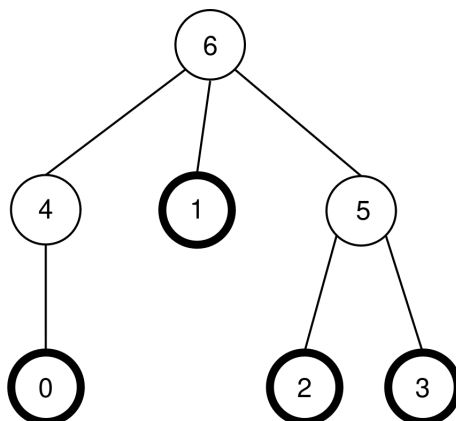
Omejitve	Točke
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Podnaloga 4 (60 točk)

Omejitve	Točke
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Primer

Razmislimo o istem stroju, ki je opisan v nalogi, torej $N = 7$, $M = 4$ in $P = [4, 6, 5, 5, 6, 6]$. Stroj je ponovno prikazan na spodnji sliki:



Ocenjevalnik izvede naslednji klic:

```
find_structure(4)
```

Ta funkcija lahko izvede naslednje zaporedje klicev:

1. **insert(0, 0)**: list 0 je prost, zato kroglico vrednosti 0 postavimo na list 0. Vozlišče $P[0] = 4$ je prosto, zato kroglica preide v vozlišče 4. Vozlišče $P[4] = 6$ je prosto, zato kroglica preide v vozlišče 6. Ker je vozlišče 6 koren, se premikanje ustavi. Klic vrne `true`.
2. **insert(3, 20)**: list 3 je prost, zato kroglico vrednosti 20 postavimo na list 3. Vozlišče $P[3] = 5$ je prosto, zato kroglica preide v vozlišče 5. Ker je $P[5] = 6$ zasedeno, se premikanje ustavi. Klic vrne `true`.

3. `insert(1, 10)`: list 1 je prost, zato kroglico vrednosti 10 postavimo na list 1. Ker je $P[1] = 6$ zasedeno, kroglica ostane na vozlišču 1. Klic vrne `true`.
4. `insert(2, 30)`: list 2 je prost, zato kroglico vrednosti 30 postavimo na list 2. Ker je $P[2] = 5$ zasedeno, kroglica ostane na vozlišču 2. Klic vrne `true`.
5. `insert(1, 25)`: list 1 je zaseden, zato kroglice ne uspemo postaviti na list 1. Klic vrne `false`.
6. `insert(0, 20)`: list 0 je prost, zato kroglico vrednosti 20 postavimo na list 0. Vozlišče $P[0] = 4$ je prosto, zato kroglica preide v vozlišče 4. Ker je $P[4] = 6$ zasedeno, se premikanje ustavi. Klic vrne `true`.

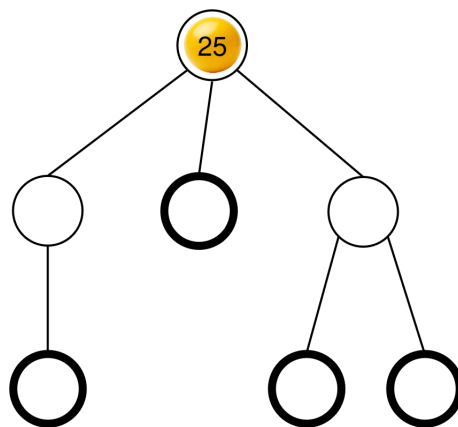
Konfiguracija kroglic, ki je posledica tega dogajanja, je bila že prikazana v opisu naloge.

Nato lahko funkcija pokliče:

```
collect()
```

Izvedba tega klica je bila že razložena v opisu naloge, torej ta klic vrne $S = [0, 10, 20, 30, 20]$. Po tem je stroj ponovno prazen.

Nato lahko funkcija pokliče `insert(2, 25)`. List 2 je prost, zato kroglico vrednosti 25 postavimo na list 2. Ta kroglica nato preide v vozlišče 5, nato pa v vozlišče 6, kjer se ustavi. Klic vrne `true`. Konfiguracija kroglic, ki je posledica tega dogajanja, je prikazana na naslednji sliki.



Na koncu lahko funkcija pokliče `collect()`, ki vrne $S = [25]$ in izprazni stroj.

Obstajata dve pravilni polji staršev, ki bi ju `find_structure` lahko vrnila: $R = P = [4, 6, 5, 5, 6, 6]$ (ustreza označevanju $L = [0, 1, 2, 3, 4, 5, 6]$) in $R = [5, 6, 4, 4, 6, 6]$ (ustreza označevanju $L = [0, 1, 2, 3, 5, 4, 6]$). V tem primeru je $K = 2$ in $B = 30$, torej $C = 32$.

Vzorčni ocenjevalnik

Oblika vhoda

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Oblika izhoda

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Tukaj je Q dolžina polja R , ki ga vrne `find_structure`.